



0x4E7a3b2F8c1d9E0a...	SHA256: ...	SHA256: d...
0x4E7a3b2F8c1d9E0a...	SHA256: ...	SHA256: 5...
0x4E7a3b2F8c1d9E0a...	SHA256: ...	SHA256: 4...
0x4E7a3b2F8c1d9E0a...	SHA256: ...	SHA256: c...
0x4E7a3b2F8c1d9E0a...	SHA256: ...	SHA256: 0...
0x4E7a3b2F8c1d9E0a...	SHA256: ...	SHA256: 1...
...	...	...
0x4E7a3b2F8c1d9E0a...	SHA256: ...	SHA256: 0...
0x4E7a3b2F8c1d9E0a...	SHA256: ...	SHA256: ...

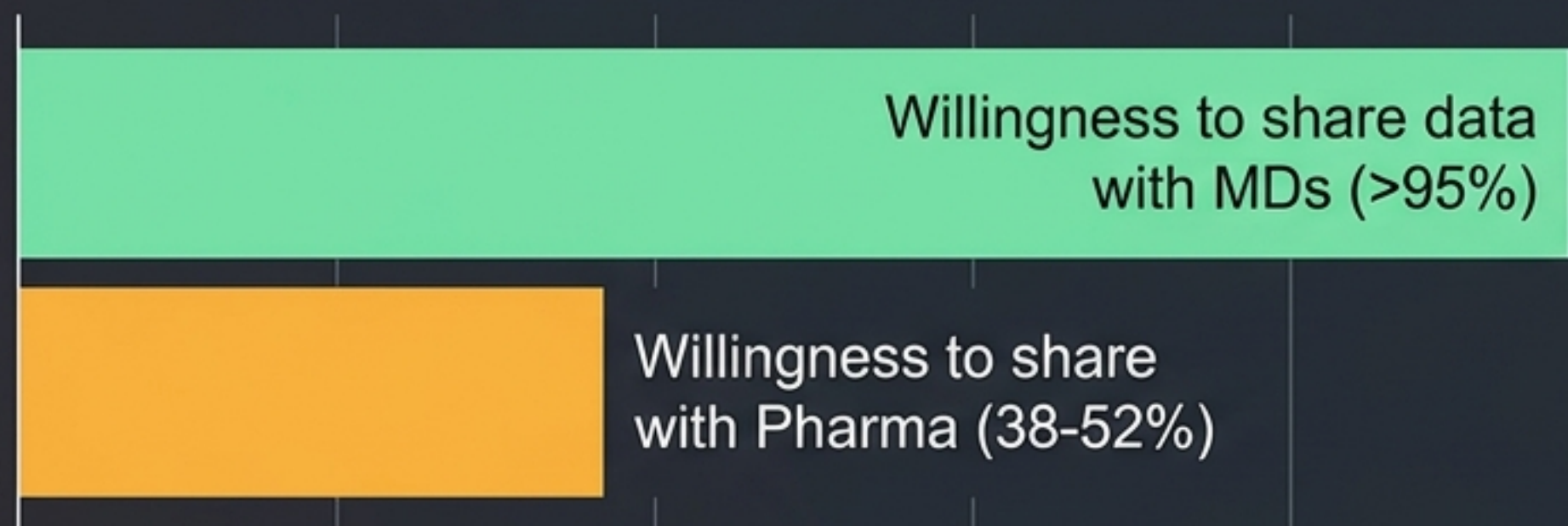
VitaPod System Architecture

Privacy-Preserving Clinical Trial Eligibility via zkML

> _

The Cost of the Privacy Gap in Clinical Trials

The Problem



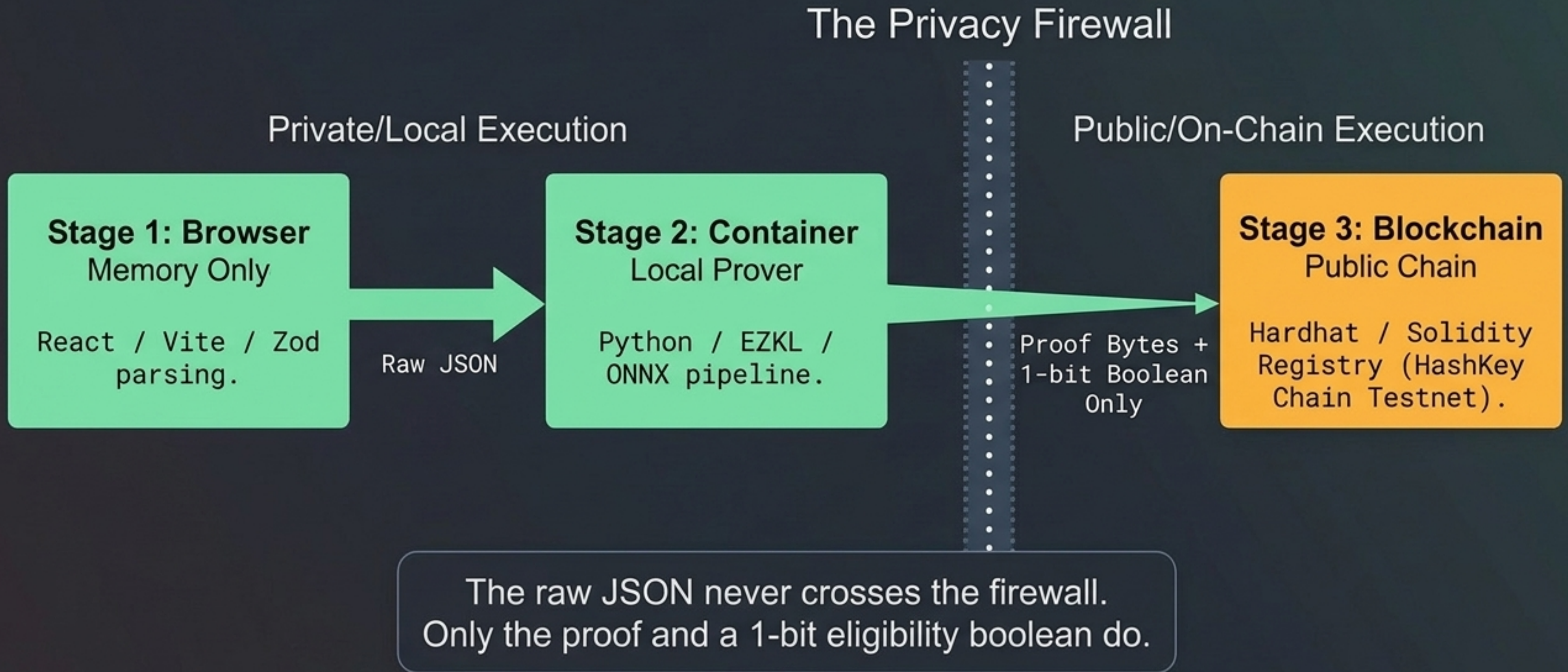
- 40% of NCI network trials fail to complete accrual.
- Screen failures cost \$800–\$2500 per patient due to this data willingness gap.

(Source: Cascini et al., 2024).

Engineering Constraints

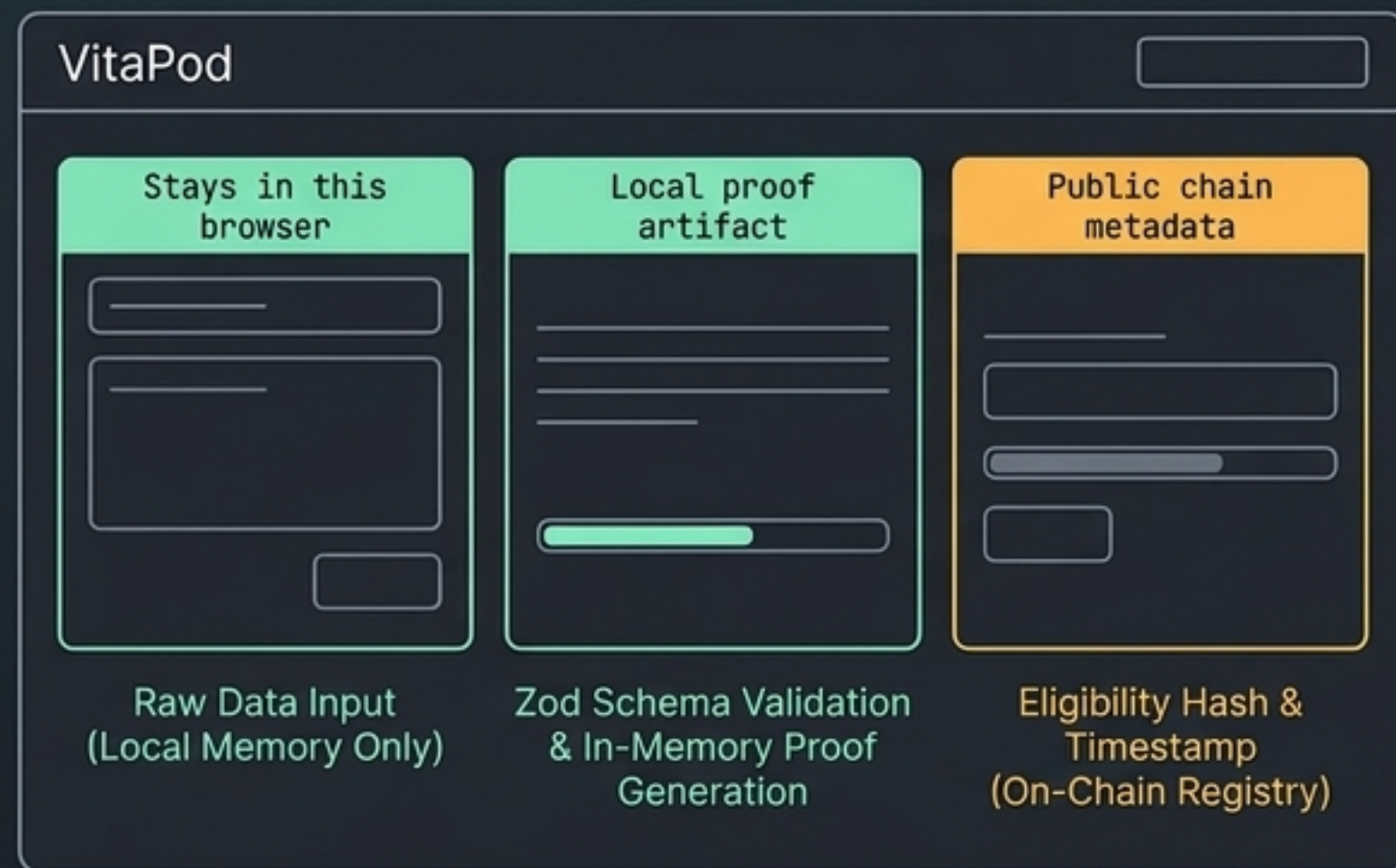
- [x] 100% Local Execution (Zero-trust)
- [x] Cryptographic Verifiability (zk-SNARKs)
- [x] Deterministic Reproducibility (Docker-first)

High-Level Architecture: The 3-Stage Air-Gap



Stage 1: Frontend Guardrails & In-Memory Execution

Tech Stack: React + Vite + Zod. Single Source of Truth: `packages/shared` strict schema validation (`vitapod.synthetic-lab.v1`).



```
no-upload.test.ts (Static Guard)

// no-upload.test.ts (Static Guard)
expect(source).not.toMatch(/\b(fetch|axios|XMLHttpRequest)\b/);
expect(source).not.toMatch(/\b(localStorage|indexedDB)\b/);
```

Engineering Highlight

Instead of trusting the UI component state, CI unit tests statically scan the React source code to mathematically guarantee network persistence APIs are never imported or used.

Stage 2: The EZKL zkML Proving Pipeline



Ingests
`synthetic-lab.v1`
fixture.

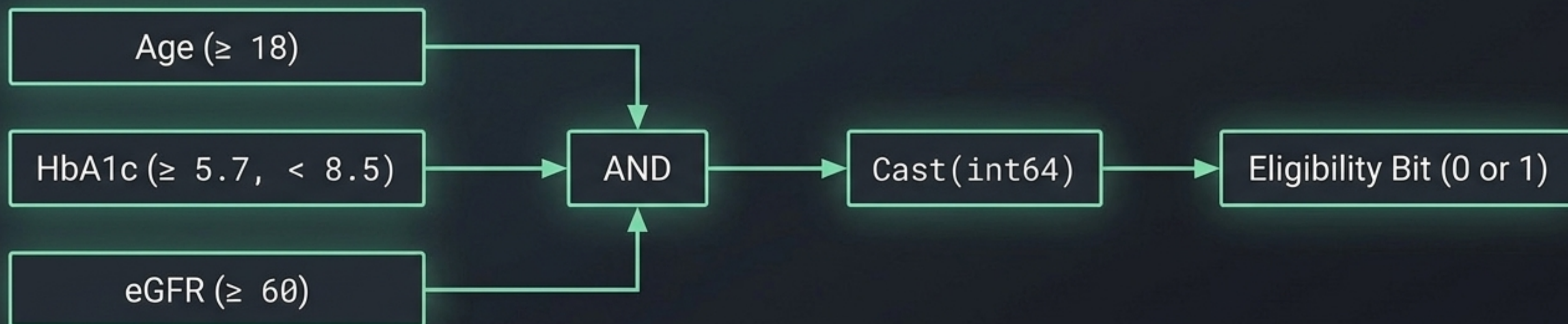
Floating point
normalization to
fixed-point integers
(e.g., HbA1c 5.7 -> 57).
Circuit requires exact
integers to prevent
quantization drift.

EZKL v23.0.5
compiling the direct
ONNX model.

A sanitized bundle
containing the proof
bytes, public instances,
and model hashes
(`modelSha256:`
`abeff1...`).

The Deterministic Policy Graph

Private/Local Execution



Architecture Insight

A fixed, hand-crafted ONNX policy (`onnx.helper.make_node`) was chosen over a trained PyTorch ML model for the prototype. This byte-deterministic threshold logic proves the ZK primitive works flawlessly before introducing the complexities of floating-point quantization errors.

The Cryptographic Data Boundary



Strictly Private (Device-Bound)

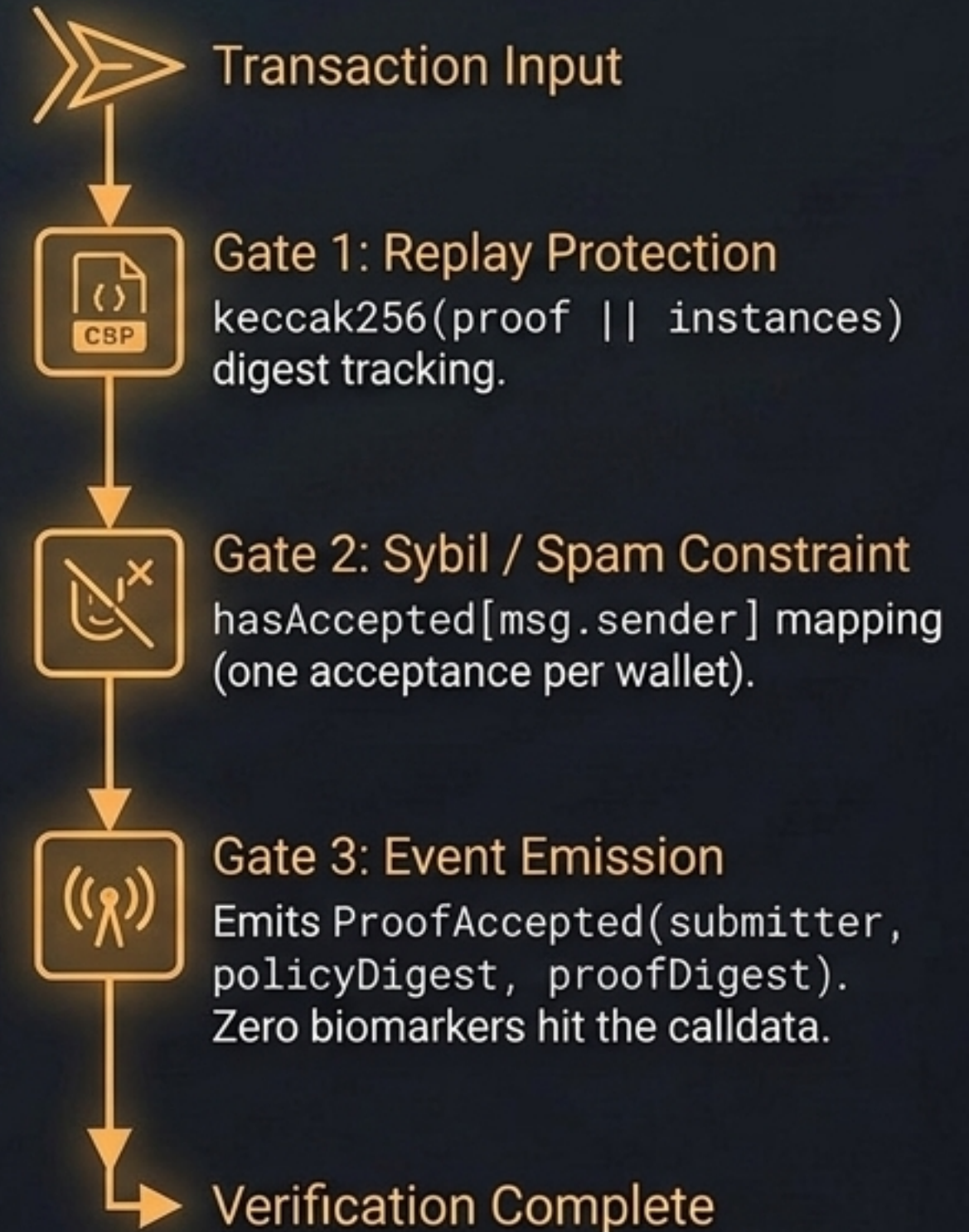
- ✓ Raw biomarkers (Age, HbA1c, eGFR)
- ✓ Normalized integer inputs (`input_data`)
- ✓ The Witness parameters
- ✓ The EZKL Proving Key (`pk.key`)



Public On-Chain (Sanitized)

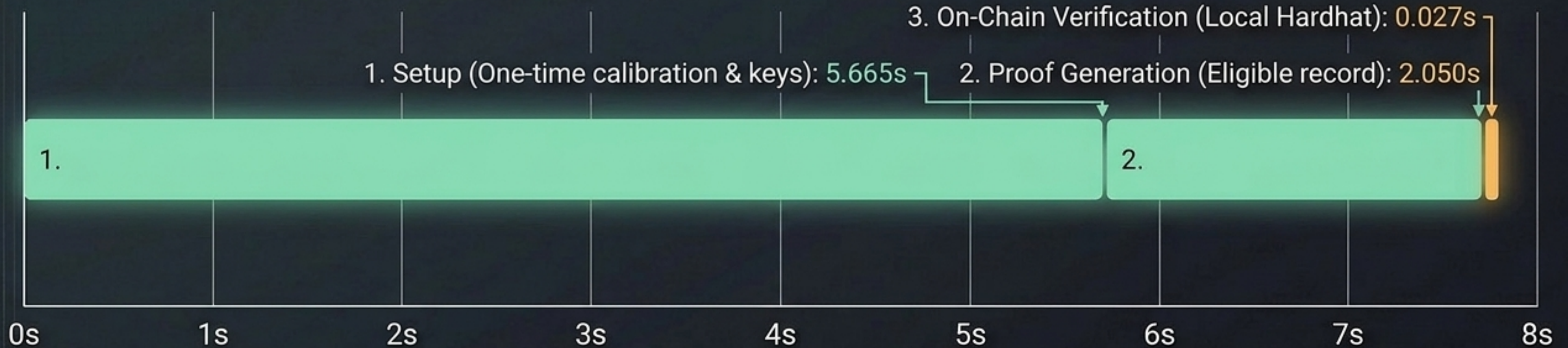
- The Eligibility Result (1 bit)
- Ha1o2 Proof Bytes (`0x0102...`)
- Submitting Wallet Address
- Model & Policy Digests (keccak256)

Stage 3: On-Chain Verification Architecture



Native Execution & Proving Latency

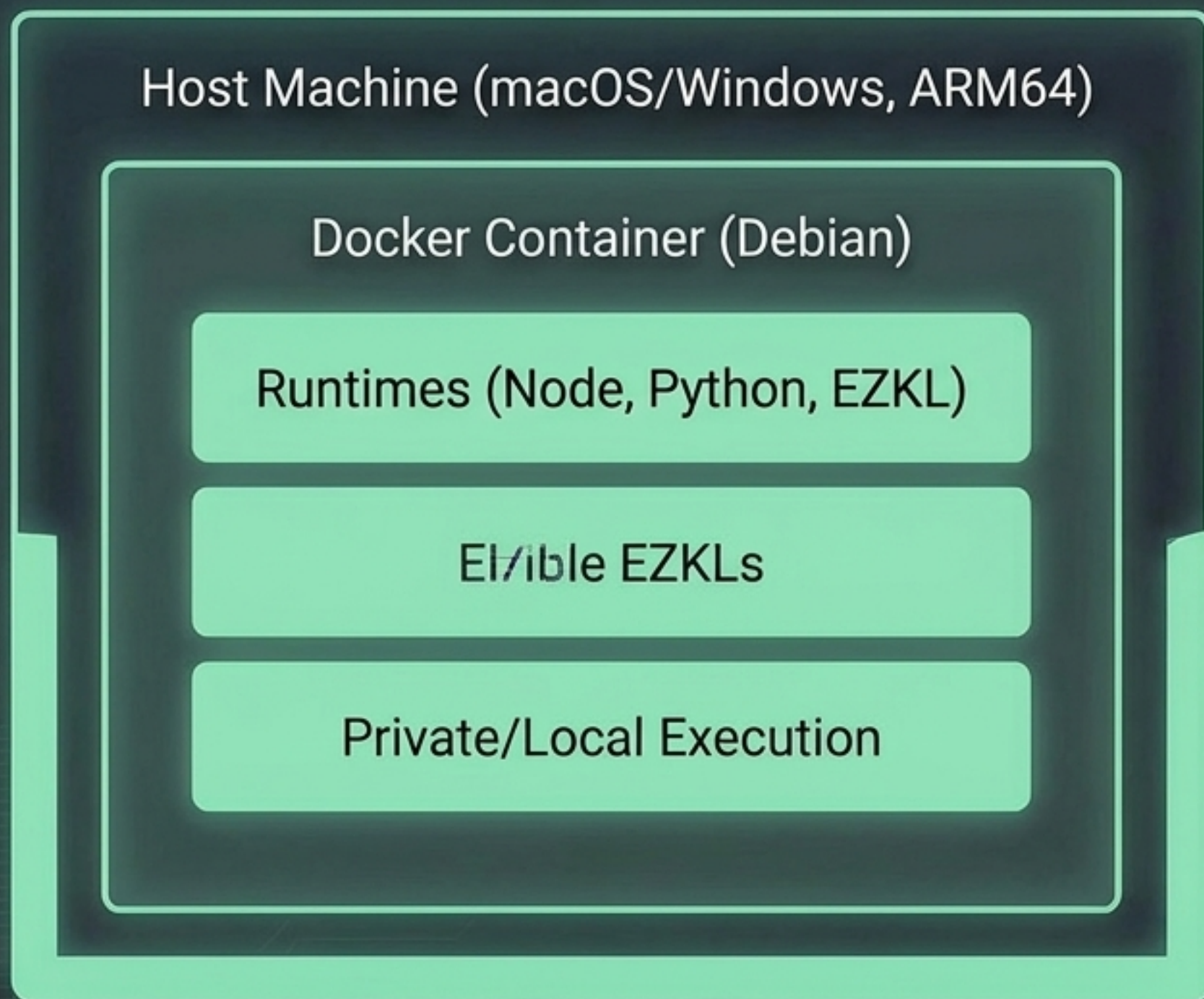
Measured natively on ARM64 architecture (Apple Silicon) inside the pinned container.



Key Insight

A 2.05-second proof generation is fast enough for synchronous, interactive UI flows without requiring complex background workers or loaders.

Reproducibility & Infrastructure Topology



The Challenge: zkML toolchains are notoriously fragile and highly sensitive to OS and interpreter environments.

The Solution: A 100% containerized, deterministic toolchain.

Strict Pins:

- Base image: `node:22.23.1-bookworm@sha256...`
- Python: `3.11.2`
- EZKL: `v23.0.5` (binary hash checked at runtime).

Result: A reviewer needs only Docker on their host to run **lint**, **pytest**, TS proving, and the Hardhat node via `demo-smoke.sh`.

Architectural Trade-offs & Decisions

DECISION POINT	CHOSEN PATH	RATIONALE & CONTEXT
Browser WASM vs. CLI Prover	Chosen: CLI Prover in Docker.	Why: Pinned EZKL v23.0.5 lacked WASM assets. 157MB proving keys and strict COOP/COEP headers made in-browser proving infeasible for the timebox.
PyTorch export vs. Direct ONNX construction	Chosen: Direct ONNX via onnx package.	Why: Yields byte-deterministic output and a drastically leaner container image size compared to PyTorch.
Native arm64 vs. Emulated amd64 container	Chosen: Native linux/arm64.	Why: Emulation penalty inflated proving times from 2s to tens of seconds, killing the live 3-minute demo UX.

Threat Model & System Boundaries



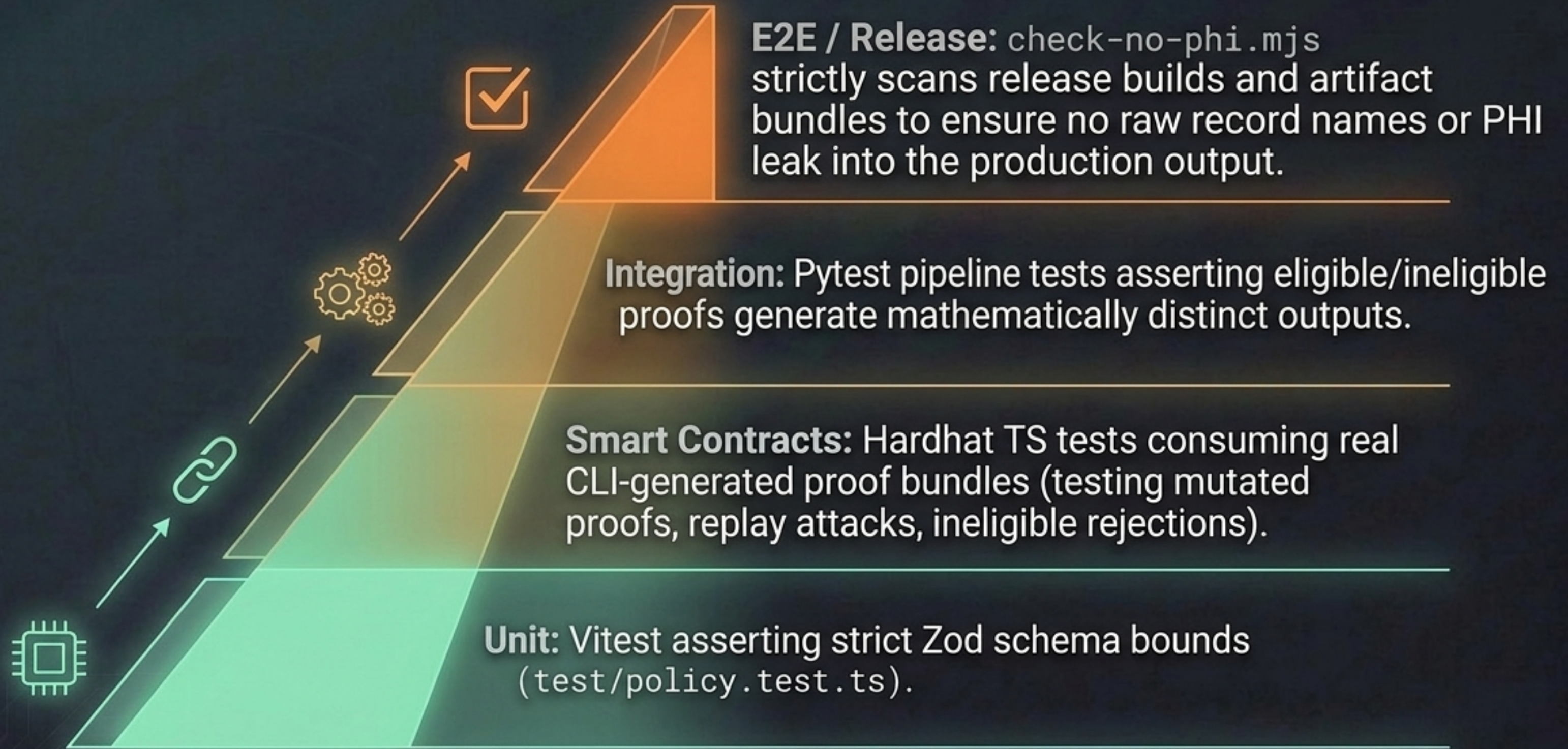
What is **Proven (In Scope)**:

- Correct, deterministic execution of a specific policy over a private synthetic witness.

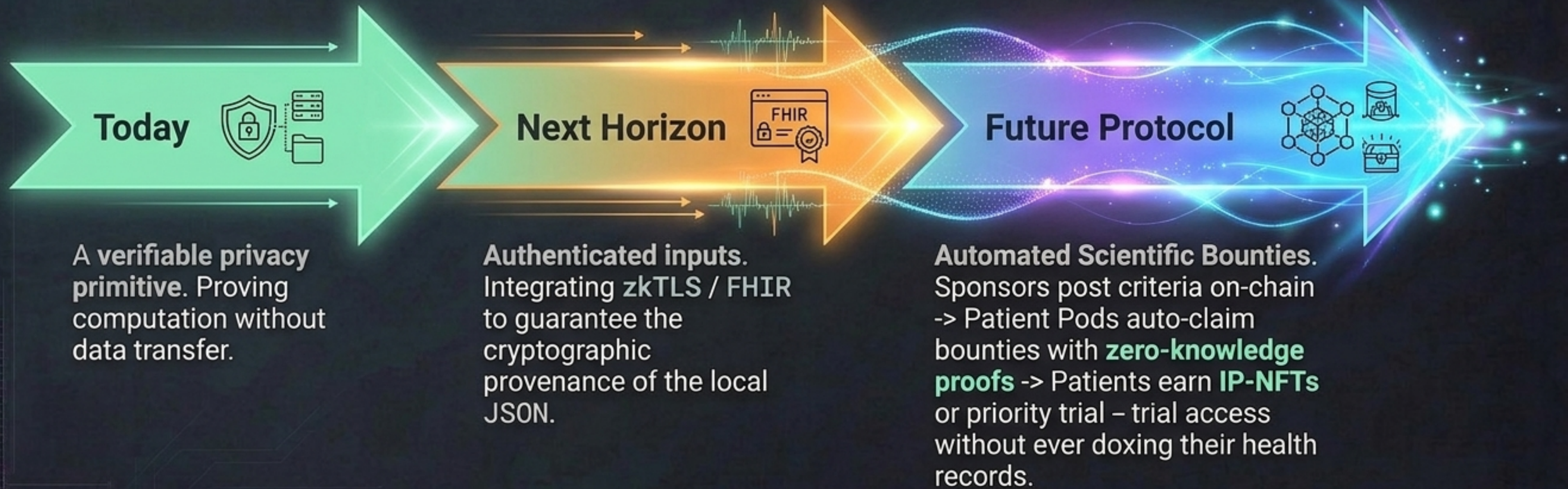
What is **NOT Proven (Out of Scope)**:

- **Provenance**: Cannot prove data came from a real lab/EHR (Requires future zkTLS/TLSNotary integration).
- **Identity**: A wallet address does not equal human identity (vulnerable to Sybil/sharing).
- **Regulatory**: Not HIPAA/FDA compliant (synthetic data only; not a medical device).

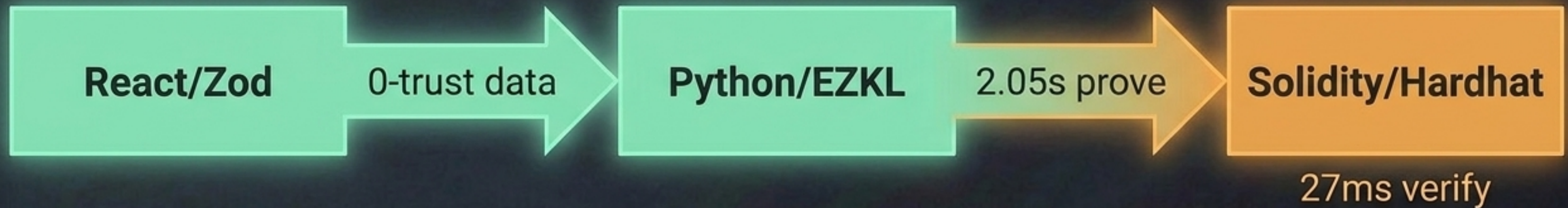
Multi-Layer QA & Testing Strategy



System Synthesis & The DeSci Future



Architecture Summary & Final Takeaway



“See the computation. Keep the record.”

We bridge the 40% clinical trial recruitment gap not by asking patients to trust sponsors more, but by changing the architecture so they don't have to trust them at all.

github.com/dyrtyData/VitaPod